Online Voting System

Group Members:

Muhammad Saad (242370092)

Muhammad Ayaan Qadir (242370045)

Muhammad Abdullah (231980037)

Abdul-Rehman Cheema (231370273)

Project Description:

This project is a simple Java-based e-voting system that allows voters to register and cast their votes for candidates. It includes two main user roles: voters and faculty. Voters can view candidates, vote once, and see voting history. Faculty members can manage candidates, reset all votes, and sort candidates by name, ID, or number of votes. The system uses basic object-oriented principles and data structures like Array-List, Stack, Queue, and Doubly Linked List to handle voting operations efficiently.

Project Objectives:

- Allow **voters** to cast, and view vote history securely.
- Enable **faculty members (admins)** to manage candidate records and control vote data.
- Maintain a **log of vote actions** using a **Doubly Linked List**.
- Implement sorting and searching operations to analyze election data.
- Use **data structures** effectively to store and manipulate system data.

Core Features & Functionalities:

👤 User Roles:

1. **Voter**:
    - Registers and logs into the system.
    - Casts a vote (only once).
    - Can **undo** the last vote.
    - Sees list of candidates.
    - Views **voting history**.
2. **Faculty/Admin**:
    - Adds/removes candidates.
    - **Resets all votes**.
    - Sorts candidates by **Name, ID, or Votes**.
    - Views **Top N Candidates**.
    - Has access to advanced analytics tools.

Key Classes & Responsibilities:

Class	Functionality
`Person`	Base class with name, age, email, and validations.
`User`	Extends `Person`, adds user ID and common methods.
`Voter`	Extends `User`, manages voting status and ID format validation.
`Faculty`	Admin role, can add/remove/sort/reset candidates and votes.
`Candidate`	Represents a candidate, includes vote tally and merge sort logic.
`E-Voting-System`	Main logic: voter registration, vote casting, undoing, and history tracking.
`Vote-History`	Implements a **doubly linked list** for tracking vote actions.

Data Structures Used:

Data Structure	Used In	Purpose
`Array-List`	Voters, Candidates	Dynamic, indexed list operations.
`Stack<String>`	Undo system	LIFO undo of vote actions.
`Queue<String>`	Vote logs	FIFO logging of vote events.
`Doubly Linked List`	`Vote-History`	Forward and backward navigation of vote history.
`Map<String, Voter>`	Admin reset logic	Quick access to voter records by ID.

Algorithms Implemented:

Algorithm	Purpose	Used In
Merge Sort	Sort candidates by votes (high to low)	`Candidate.mergeSortByVotes()`
Selection Sort	Find Top-N Candidates	`Faculty.getTopCandidates()`
Insertion Sort	Sort by name (A-Z)	`Faculty.sortCandidatesByName()`
Bubble Sort	Sort by ID	`Faculty.sortCandidatesById()`
Palindrome Check	Check if a candidate's agenda is a palindrome	`Candidate.isAgendaPalindrome()`

Future Scope:

- Integration with biometric or OTP-based verification.
- Blockchain-based vote integrity.
- Deployment as a web or mobile app.
- Live vote count dashboard with real-time updates.